

无哈希函数下的哈希表设计综述

巨文博

(南京大学智能软件与工程学院 苏州 215000)

摘 要 哈希表是字典问题中最常用的随机化数据结构之一, 通常依赖哈希函数将键映射到存储位置。然而, 当希望获得极低失败概率时, 传统构造会受到显式哈希函数族能力的限制。Kuszmaul 在论文“A Hash Table Without Hash Functions, and How to Get the Most Out of Your Random Bits”中提出了一种不同思路: 不再直接使用哈希函数, 而是将随机性嵌入到数据结构内部。本文围绕该论文的主要结构展开综述, 重点介绍 rotated radix trie 的基本思想、amplified rotated trie 降低失败概率的方法, 以及 budget rotated trie 压缩随机比特数量的思路。

关键词 哈希表; 随机化字典; 旋转基数树; 失败概率; 随机比特

1 引言

字典(dictionary)是一类支持插入、删除和查询操作的数据结构。在算法设计中, 随机化字典通常也被称为哈希表。经典哈希表通过哈希函数将键映射到桶或槽位中, 从而在期望意义或高概率意义下实现常数时间操作。

本文综述的对象是 Kuszmaul 提出的无哈希函数哈希表构造^[1]。该论文关注的问题是: 在存储 n 个 $\Theta(\log n)$ 位键值对、使用线性空间并支持常数时间操作的前提下, 哈希表能够提供多强的概率保证。

传统研究通常将概率保证与哈希函数族联系在一起。如果能够使用完全随机哈希函数, 则可以得到非常强的负载均衡效果; 但完全随机函数难以显式高效实现。另一方面, 已知的常数时间显式哈希函数族往往只能提供有限独立性, 从而限制了失败概率的进一步降低。

Kuszmaul 的核心思路是绕开这一瓶颈: 构造一种具有哈希表功能的随机化字典, 但不再直接使用哈希函数。随机性不表现为 $h(x)$ 这样的键到桶的映射, 而是以节点数组随机旋转的形式嵌入到数据结构内部。

2 研究背景

2.1 字典与失败概率

一个字典需要维护一个动态集合, 并支持查询某个键是否存在以及取回对应值。在随机化字典中, 所谓失败概率通常指某次操作不能在常数时间内完成的概率。当失败概率不超过 $1/\text{poly}(n)$ 时, 通常称该结构以高概率成功。

确定性的常数时间字典是一个重要问题。已有工作如动态 fusion node 可以对较小规模集合实现确定性常数时间操

作^[2], 但对于 $\Theta(n)$ 规模的一般集合, 确定性常数时间字典仍然是困难问题。

2.2 哈希函数瓶颈

过去关于高概率哈希表的研究大多依赖哈希函数族。问题在于, 如果使用完全随机哈希函数, 理论上可以得到很强的负载均衡; 但若将其替换为可显式构造、常数时间求值的哈希函数族, 概率保证往往会下降。

因此, 论文提出的方向不是继续寻找更强的传统哈希函数, 而是改变随机性的使用方式: 将随机性直接放入数据结构本身。这一点是本文后续结构的核心出发点。

3 Rotated Radix Trie

3.1 基本结构

rotated radix trie 是论文中的基础结构。普通 radix trie 可以根据键的不同片段逐层选择孩子节点, 但如果直接为每个内部节点维护一个大数组, 整体空间可能达到二次量级。

rotated radix trie 的做法是: 为每个内部节点 s 选择一个随机旋转量 r_s , 然后把各个节点数组经过循环旋转后叠加到一个总数组中。如果某个非空指针原本位于 child index c , 则旋转后映射到

$$\phi(s, c) = (c + r_s) \bmod n. \quad (1)$$

这个公式体现了该结构与传统哈希表的区别: 它没有对每个键计算哈希值, 而是对节点数组整体进行随机旋转。

3.2 Ball、Bin 与 Load

为了分析该结构, 论文将每个非空指针称为一个 ball, 将叠加后总数组中的位置称为 bin。一个 ball 可以表示为 (s, c) , 其中 s 是源节点, c 是该节点中的孩子编号。

对某个 bin j , 设 Y_j 表示映射到该 bin 的 ball 数量, 则 Y_j 就是该 bin 的 load。由于总共有 $O(n)$ 个 ball, 而每个 ball 落到固定 bin 的概率约为 $1/n$, 因此可以得到

$$E[Y_j] = O(1). \quad (2)$$

论文进一步使用 Chernoff bound 说明 Y_j 超过 $\text{polylog}(n)$ 的概率很小。只要每个 bin 的 load 被控制在 $\text{polylog}(n)$ 以内, 就可以用 dynamic fusion node 存储每个 bin 内部的小集合, 从而保持常数时间操作。

3.3 结构小结

rotated radix trie 的意义在于, 它证明了可以通过随机旋转数组而不是直接使用哈希函数来构造随机化字典。它本身已经可以达到线性空间和高概率常数时间操作, 但失败概率还不够强, 因此后续需要 amplified rotated trie 进一步放大概率保证。

4 Amplified Rotated Trie

4.1 Overflow Ball

amplified rotated trie 是 rotated radix trie 的改进版本, 目标是将失败概率降低到接近最优的量级。该结构首先允许主结构中出现少量溢出。当某个 bin 中已经存放了 $\text{polylog}(n)$ 个 ball, 而新的 ball 仍然映射到该 bin 时, 这个新的 ball 被称为 overflow ball。

overflow ball 不再直接放入原 bin, 而是存储到辅助结构 Q 中。这样主结构中每个 bin 的规模仍然可以被控制住。

4.2 控制溢出数量

辅助结构 Q 本身并不一定空间高效, 因此必须证明 overflow ball 的总数 q 足够小。论文将 q 看成所有随机旋转量的函数:

$$f(r_1, r_2, \dots, r_m) = q. \quad (3)$$

为了分析该函数的集中性, 论文使用了 McDiarmid 不等式^[3]。直观上, 如果只改变某一个节点的随机旋转量, 最多只会影响该节点产生的 ball。Section IV 将 fanout 调整为 n^δ , 使得单个随机变量对 q 的影响被限制在 n^δ 范围内, 从而满足使用 McDiarmid 不等式所需的有界差分条件。

4.3 概率保证

通过降低 fanout、引入 overflow ball、并对 overflow ball 总数进行集中性分析, amplified rotated trie 可以将失败概率降低到

$$O\left(\frac{1}{n^{1-\epsilon}}\right) \quad (4)$$

的量级。这个结果显著强于基础 rotated radix trie, 也接近理论上可以期待的最强概率保证。

5 Budget Rotated Trie

budget rotated trie 是论文的另一个方向。与 amplified rotated trie 追求极低失败概率不同, budget rotated trie 更关注随机比特数量, 希望在保持 $1/\text{poly}(n)$ 失败概率的同时, 只使用约 $O(\log n \log \log n)$ 个随机比特。

为了减少随机比特的使用, budget rotated trie 不再为大量内部节点直接生成完全独立的随机旋转量。一个基本做法是减少真正需要显式创建的内部节点数量: 当某个子树规模较小时, 可以先用 dynamic fusion node 直接存储, 只有当子树规模超过阈值后才创建新的内部节点。

在此基础上, 论文进一步改变 ball 到 bin 的映射方式。基础 rotated trie 使用 $\phi(s, c) = (c + r_s) \bmod n$, 而 budget rotated trie 将全部 bin 分为 n^δ 个组, 每组大小为 $n^{1-\delta}$, 并使用

$$\psi(s, c) = ((c + a_s) \bmod n^\delta) \cdot n^{1-\delta} + b_s \quad (5)$$

进行映射。其中 a_s 用来决定 ball 被分配到哪个组, b_s 用来决定它在组内的具体位置。这样的拆分使得两类随机参数可以用不同方法生成: a_s 可由常数独立哈希函数产生, 用于控制各组中的 ball 数量; b_s 则由具有负载均衡性质的 gradually-increasing-independence hash functions 产生, 用于控制组内最大负载。

gradually-increasing-independence hash functions 的直接求值时间并非常数, 但在 budget rotated trie 中, 它们主要作为生成随机参数的工具。由于新的内部节点只会对应于子树规模达到 $\text{polylog}(n)$ 后创建, 相关初始化成本可以被摊销到之前的插入过程中。这样一来, 结构仍然能够维持高概率常数时间操作, 同时显著减少随机比特使用量。

参考文献

- [1] KUSZMAUL W. A hash table without hash functions, and how to get the most out of your random bits[C]//2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS). IEEE, 2022: 991-1001.

-
- [2] PATRASCU M, THORUP M. Dynamic integer sets with optimal rank, select, and predecessor search[C]//2014 IEEE 55th Annual Symposium on Foundations of Computer Science (FOCS). IEEE, 2014: 166-175.
- [3] MCDIARMID C. On the method of bounded differences[J]. Surveys in Combinatorics, 1989, 141: 148-188.